Managing Application Access

Kubernetes Production · Lesson 10

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **Service** abstraction, selectors and **EndpointSlices**
- Choose the right **Service type**: ClusterIP, NodePort, LoadBalancer, ExternalName
- Use **headless Services** and **StatefulSet** stable DNS names
- Resolve workloads with **cluster DNS** naming
- Expose HTTP/HTTPS apps with **Ingress** and an **IngressClass**
- Distinguish **Service (L4)** from **Ingress (L7)** and use `kubectl port-forward`
- Route traffic with the **Gateway API** (GatewayClass / Gateway / HTTPRoute)

> Reference: *Service, Ingress, Gateway API, DNS for Services and Pods* — kubernetes.io/docs

10.1 The Service Abstraction

Why Services?

Pods are **ephemeral**: they are created and destroyed dynamically, and their **IPs change**.

A **Service** exposes a logical set of Pods behind a single, **stable endpoint**.

- Decouples frontend clients from the backend Pod details
- Tracks the changing set of healthy Pods automatically
- Provides a stable **virtual IP (ClusterIP)** and **DNS name**
- Load-balances connections across the matching Pods

> The Service is the **L4** building block of Kubernetes networking — it works on TCP/UDP/SCTP ports, not on HTTP paths.

Selectors & EndpointSlices

A Service with a **selector** continuously matches Pods by their **labels**.

- Matching Pod IP:port pairs are tracked in **EndpointSlices**
- EndpointSlices are populated automatically for selector-based Services
- The legacy **Endpoints** object is now **deprecated** in favour of EndpointSlices
- A Service **without a selector** needs manually managed EndpointSlices
- **kube-proxy** programs the rules that map the virtual IP to those Pods

```
kubectl get endpointslices
kubectl describe service my-service
```

Defining a Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80          # port exposed by the Service
      targetPort: 9376  # port on the Pods (defaults to port)
```

- `port` — the port clients use on the Service's ClusterIP
- `targetPort` — the container port; may reference a **named** port
- Omitting `targetPort` makes it default to the value of `port`

Multi-Port Services

A single Service can expose **several named ports**, each to its own targetPort.

```
spec:
  selector:
    app: web
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 8080
    - name: https
      protocol: TCP
      port: 443
      targetPort: 8443
```

- Each entry **must** be named when more than one port is defined
- Different ports may use different protocols (TCP, UDP, SCTP)

10.2 Service Types

The Four Service Types

Type	Reachable from	Builds on
ClusterIP *(default)*	Inside the cluster only	—
NodePort	`<NodeIP>:<port>` externally	ClusterIP
LoadBalancer	Cloud provider's external LB	NodePort + ClusterIP
ExternalName	CNAME to an external DNS name	— (no proxying)

- Higher types **layer on** the lower ones (LoadBalancer ⊃ NodePort ⊃ ClusterIP).
- **kube-proxy** on each node programs the rules that route the virtual IP to Pods.

> CKA tip: most workloads stay **ClusterIP**; you expose them externally with a single **Ingress** or LoadBalancer at the edge.

ClusterIP & NodePort

ClusterIP (default)

- Allocates a stable cluster-internal virtual IP
- Only reachable from inside the cluster
- The foundation every other type relies on

NodePort

- Opens the **same static port on every node**, forwarding to the Service
- Default port range: **30000–32767**
- Reachable externally at `<NodeIP>:<NodePort>`; also gets a ClusterIP

```
kubectl expose deployment web --type=NodePort --port=80 --target-port=8080
```

LoadBalancer & ExternalName

LoadBalancer

- Provisions an **external load balancer** via the cloud provider
- Automatically chains the NodePort and ClusterIP underneath
- On bare metal you need an add-on (e.g. MetalLB) to fulfil it

ExternalName

- Maps the Service to an external **DNS name** via a CNAME record
- No selector, no proxying, no EndpointSlices

```yaml
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  type: ExternalName
  externalName: db.prod.example.com
```

10.3 Headless Services & DNS

Headless Services

Set `clusterIP: None` to create a **headless** Service (no virtual IP).

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  clusterIP: None
  selector:
    app: nginx
  ports:
    - port: 80
```

- DNS returns the **individual Pod IPs** (A/AAAA records), not one VIP
- Clients select or round-robin Pods directly — ideal for **StatefulSets**
- Without a selector, you manage the EndpointSlices yourself

Cluster DNS Naming

Every Service gets a DNS name resolved by **CoreDNS**:

```
<service>.<namespace>.svc.cluster.local
```

- Same namespace: just use `<service>` (e.g. `my-svc`)
- Other namespace: `<service>.<namespace>` (e.g. `my-svc.prod`)
- A Pod's `/etc/resolv.conf` adds search domains, so short names expand
- Named ports also publish **SRV** records: `_<port>._<proto>.<svc>.<ns>.svc...`

> A normal Service resolves to its **ClusterIP**; a **headless** Service resolves to the **set of Pod IPs**.

StatefulSet & Pod DNS

A **StatefulSet** + headless Service gives each Pod a **stable** DNS name:

```
<pod-name>.<service>.<namespace>.svc.cluster.local
web-0.nginx.default.svc.cluster.local
```

- Names **persist** across reschedules — essential for clustered databases
- Pods also get IP-based records: `<ip-dashes>.<namespace>.pod.cluster.local`
  - e.g. `172-17-0-3.default.pod.cluster.local`

```
kubectl run -it --rm dnsutils --image=busybox:1.28 \
  -- nslookup web-0.nginx.default.svc.cluster.local
```

10.4 Ingress (L7 HTTP Routing)

What Is Ingress?

Ingress exposes **HTTP/HTTPS** routes from outside the cluster to Services, providing:

- Name-based **virtual hosting** (route by hostname)
- **Path-based** routing to different backend Services
- **TLS termination** and load balancing at L7

> An Ingress resource does **nothing on its own** — you must deploy an **Ingress controller** (nginx, HAProxy, cloud LB) to fulfil it.

- For non-HTTP traffic, use a NodePort or LoadBalancer Service instead.
- The Ingress API is **frozen**; the project now recommends the **Gateway API**.

A Minimal Ingress

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

- An Ingress must define `spec.rules` and/or a `spec.defaultBackend`.

IngressClass & Rules

IngressClass selects which controller handles the resource.

- Set via `spec.ingressClassName`
- If omitted, the cluster's **default** IngressClass is used (if one exists)

Each HTTP rule combines:

- **host** *(optional)* — e.g. `foo.bar.com`; if omitted, matches all hosts
- **path** — the URI to match, e.g. `/app`
- **backend** — the target `service.name` + `service.port.number`

> The **defaultBackend** serves any request that matches no rule (required if no rules are specified).

pathType Matters

Every path **must** declare a `pathType`:

pathType	Matching behaviour
Exact	Matches the URL path exactly — `/foo` ≠ `/foo/`
Prefix	Matches by `/`-split path prefix — `/foo` matches `/foo/bar`
ImplementationSpecific	Delegated to the IngressClass / controller

```
- path: /api
  pathType: Prefix
  backend:
    service:
      name: api-svc
      port:
        number: 8080
```

> CKA tip: omitting `pathType` is invalid in `networking.k8s.io/v1`.

10.5 Service vs Ingress & port-forward

Service (L4) vs Ingress (L7)

Aspect	Service	Ingress
Layer	L4 (TCP/UDP/SCTP)	L7 (HTTP/HTTPS)
Routes by	IP + port	Hostname + URL path
Protocols	Any	HTTP/HTTPS only
Needs controller?	No (kube-proxy)	**Yes** — an ingress controller
Typical use	Internal connectivity, NodePort/LB edge	Web traffic, virtual hosts

- Ingress routes incoming HTTP to **Services**, which forward to **Pods**.
- A single Ingress + controller can front **many** Services on one IP.

kubectl port-forward

For quick, **local** access to a Pod or Service — no Service exposure needed.

```
# Forward local port 8080 to a Pod's port 80
kubectl port-forward pod/web-0 8080:80

# Forward to a Service (picks a backing Pod)
kubectl port-forward service/my-service 8080:80
```

- Traffic tunnels through the **API server** to the target
- Great for **debugging** and reaching ClusterIP-only workloads
- It runs only while the command is active — **not** a production exposure

> CKA tip: use `port-forward` to test a Pod before wiring up a Service or Ingress.

10.6 Gateway API

Gateway API — Ingress's Successor

The **Gateway API** (`gateway.networking.k8s.io`) is the next-generation, role-oriented standard for managing traffic into the cluster. The **Ingress API is frozen** — new routing features now land here.

- More **expressive** than Ingress: header/method matching, traffic splitting, cross-namespace routing
- **Role-oriented** resources separate infrastructure from application concerns
- Handles **HTTP, HTTPS, TLS, TCP and gRPC** — not just HTTP/S
- Shipped as **CRDs** (installed separately) plus a controller that implements them

> Like Ingress, a Gateway does nothing without a **controller** (an implementation such as NGINX, Istio, Contour or Cilium).

The Resource Model

Three core kinds, each owned by a different role:

Kind	Owned by	Purpose
GatewayClass	Infra provider	The implementation/controller (like an IngressClass)
Gateway	Cluster operator	A traffic entry point: listeners, ports, protocols
HTTPRoute	App developer	How requests map to backend Services

- A **Gateway** references a **GatewayClass**; **Routes** attach to a Gateway via `parentRefs`.
- Route kinds: **HTTPRoute**, **GRPCRoute**, **TLSRoute**, **TCPRoute**, **UDPRoute**.

Gateway + HTTPRoute

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
spec:
  gatewayClassName: example-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: example-route
spec:
  parentRefs:
  - name: example-gateway
  hostnames: ["www.example.com"]
  rules:
  - matches:
    - path: { type: PathPrefix, value: / }
    backendRefs:
    - name: example-svc
      port: 80
```

Ingress vs Gateway API

Aspect	Ingress	Gateway API
API group	`networking.k8s.io`	`gateway.networking.k8s.io`
Status	**Frozen** (no new features)	**Active** — where features land
Model	One object + annotations	**Role-oriented**: Class / Gateway / Route
Protocols	HTTP/HTTPS	HTTP, gRPC, TLS, TCP, UDP
Expressiveness	Host + path	Headers, methods, weights, cross-NS
Install	Built-in API + controller	**CRDs** + controller

> CKA tip: know the three kinds and that routes attach via `parentRefs`; the API is delivered as **CRDs** you install first.

Key Takeaways

- A **Service** gives Pods a stable VIP + DNS name; selectors populate **EndpointSlices** (Endpoints is deprecated)
- **Types**: ClusterIP (internal), NodePort (`30000–32767`), LoadBalancer (cloud LB), ExternalName (CNAME)
- **Headless** (`clusterIP: None`) returns Pod IPs; StatefulSet Pods get stable `<pod>.<svc>.<ns>.svc.cluster.local` names
- DNS pattern: **`<svc>.<ns>.svc.cluster.local`**, resolved by CoreDNS
- **Ingress** = L7 HTTP routing; needs an **ingress controller** + a `pathType`
- **Service is L4, Ingress is L7**; `kubectl port-forward` is for local access
- The **Gateway API** (`gateway.networking.k8s.io`) is Ingress's role-oriented successor — **GatewayClass** → **Gateway** → **HTTPRoute**, installed as CRDs

End of Lesson 10

Next: Lesson 11 — Configuration & Quotas

`kubectl expose` · `kubectl get svc,endpointslices` · `kubectl get ingress` · `kubectl port-forward`